

Relazione Progetto 2 MCdS

Andrea Coldani (894684) Fabio Colonetti (899689)
Francesco Fusi (899741)

Giugno 2026

1 Introduzione

L'elaborato descrive lo sviluppo e l'analisi di un sistema software dedicato alla compressione di immagini in toni di grigio. L'obiettivo primario del progetto consiste nello studio e nell'applicazione della Trasformata Discreta del Coseno bidimensionale (DCT2) all'interno di un ambiente di sviluppo open-source, analizzando gli effetti di un algoritmo di compressione di tipo JPEG (operante senza matrice di quantizzazione) sul dominio frequenziale delle immagini.

Il lavoro si articola in due fasi principali: una prima parte di carattere prettamente computazionale, focalizzata sulla progettazione “fatta in casa” della trasformata e sul confronto delle sue prestazioni temporali rispetto alle routine ottimizzate di libreria basate su algoritmi *Fast* (FFT); e una seconda parte applicativa, orientata alla scomposizione dell'immagine bitmap in macro-blocchi spaziali e al successivo taglio selettivo delle alte frequenze regolate da una soglia intera d . Oltre alla validazione matematica del codice tramite specifici casi test forniti, la relazione offre un commento critico sull'efficienza degli algoritmi e sul compromesso tra il livello di compressione applicato e la qualità visiva finale delle immagini ricostruite.

2 Struttura della libreria

Il software è stato sviluppato in linguaggio Python e strutturato in modo modulare per garantire separazione tra la logica di calcolo matematico, l'interfaccia utente, i test di validazione. Di seguito viene descritto il ruolo di ciascuna cartella e file presenti nella radice del progetto:

- **data/**: Cartella contenente le immagini in formato bitmap (**.bmp**) in toni di grigio, utilizzate come input per la compressione e per le elaborazioni del software.
- **src/** (Source): Contiene l'intero codice sorgente dell'applicazione, suddiviso nei seguenti sotto-moduli specializzati:

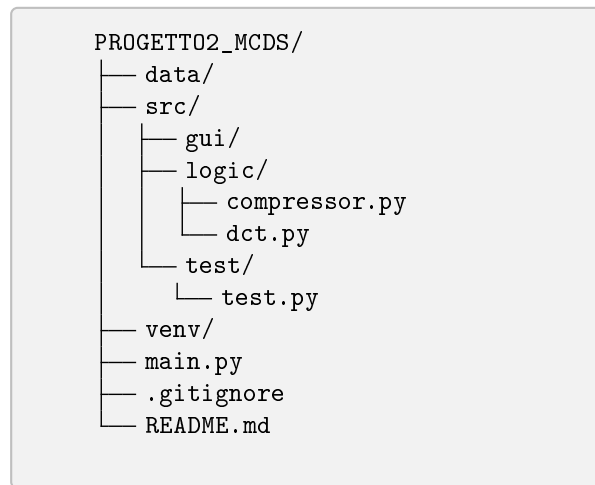


Figura 1: Struttura delle directory del progetto di compressione immagini.

- **gui/**: Gestisce l'interfaccia grafica minimale che consente all'utente di navigare nel filesystem per selezionare l'immagine e inserire i parametri numerici di input (F e d).
- **logic/**: Rappresenta il nucleo computazionale dell'algoritmo:
 - * **dct.py**: Contiene l'implementazione sequenziale della DCT/DCT2 definita "fatta in casa" ($O(N^3)$) e le funzioni di interfaccia con la libreria scientifica per la versione "fast" ($O(N^2 \log N)$), necessarie al confronto dei tempi di esecuzione.
 - * **compressor.py**: Implementa l'algoritmo di compressione JPEG-like, occupandosi della segmentazione dell'immagine in macroblocchi $F \times F$, del filtraggio delle frequenze oltre la diagonale d .
- **test/test.py**: Script di Landmark-Testing utilizzato per verificare la correttezza numerica e lo scaling delle funzioni implementate, confrontando i risultati della DCT monodimensionale e bidimensionale con la matrice di test 8×8 fornita nelle specifiche.
- **main.py**: Il punto di ingresso (*entry-point*) principale dell'applicazione che inizializza l'interfaccia utente e coordina il flusso dei dati tra i vari moduli.
- **venv/**, **.gitignore**, **README.md**: File e directory di configurazione per la gestione dell'ambiente virtuale isolato, l'esclusione di file ridondanti dal controllo di versione e la stesura delle istruzioni rapide per l'installazione e l'esecuzione del programma.

3 parte 1

3.1 Analisi delle Prestazioni della DCT2

Per comprendere le discrepanze prestazionali rilevate sperimentalmente, è necessario analizzare la logica algoritmica alla base delle due implementazioni.

L'algoritmo sviluppato manualmente applica la definizione classica della trasformata sfruttando la proprietà di *separabilità*. La DCT2 viene quindi calcolata come due trasformate monodimensionali in cascata (prima per righe e poi per colonne) tramite prodotti matrice-matrice del tipo $c = T \cdot f \cdot T^T$. Ciascun prodotto richiede l'esecuzione di cicli annidati su strutture dati di dimensione N , portando il costo computazionale teorico a un ordine di complessità pari a $\mathcal{O}(N^3)$.

Al contrario, la routine di libreria ottimizzata si appoggia ad algoritmi di tipo *Fast* correlati alla Trasformata Rapida di Fourier (FFT). Invece di eseguire il prodotto matriciale diretto, la FFT riduce drasticamente il numero di operazioni ridondanti. Sfruttando la simmetria e la ricorsività (secondo una logica *divide et impera*), il problema di dimensione N viene suddiviso in sottoproblemi più piccoli. Questo approccio abbatta la complessità computazionale del calcolo bidimensionale a:

$$\mathcal{O}(N^2 \log_2 N) \quad (1)$$

dove N^2 rappresenta il numero totale di pixel dell'immagine, mentre il fattore logaritmico descrive l'efficienza della scomposizione in frequenza.

3.1.1 Dati Sperimentali Raccolti

I test di calcolo sono stati eseguiti su array quadrati di dimensione N crescente. I tempi di esecuzione registrati per entrambe le versioni sono riassunti nella Tabella 1.

N	Manuale $\mathcal{O}(N^3)$ [s]	Libreria $\mathcal{O}(N^2 \log N)$ [s]
8	6.592274×10^{-4}	1.833677×10^{-3}
16	2.862453×10^{-3}	1.497269×10^{-4}
32	7.899284×10^{-3}	1.859665×10^{-4}
64	3.401685×10^{-2}	2.512932×10^{-4}
128	1.806502×10^{-1}	3.659725×10^{-4}
256	1.715369×10^0	1.292467×10^{-3}

Tabella 1: Confronto dei tempi di esecuzione tra algoritmo manuale e di libreria al variare di N .

3.1.2 Commento dei Risultati

L'analisi dei dati in Tabella 1 e del relativo andamento in Figura 2 convalida sperimentalmente i modelli di complessità teorica precedentemente descritti.

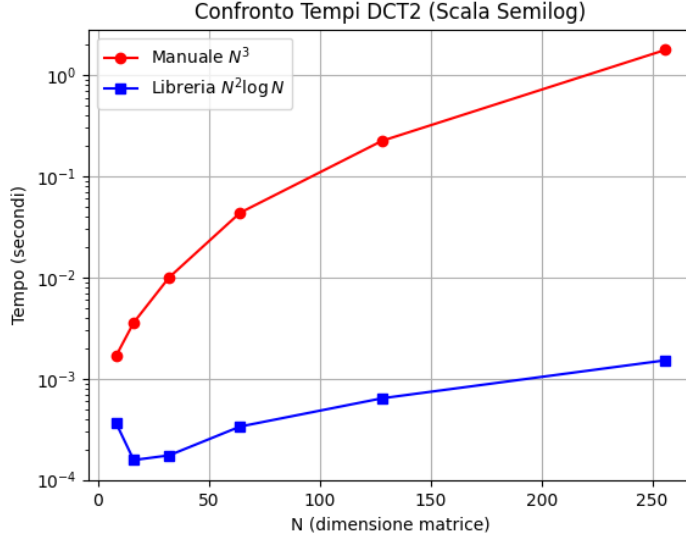


Figura 2: Andamento dei tempi di esecuzione in scala semilogaritmica al variare della dimensione N .

L'andamento del codice manuale mostra una crescita dei tempi strettamente legata al fattore N^3 . Quando la dimensione della matrice raddoppia (passando ad esempio da $N = 128$ a $N = 256$), il tempo teorico dovrebbe aumentare di un fattore pari a $2^3 = 8$. Dai dati sperimentali si osserva un incremento effettivo di:

$$\frac{1.715369}{0.1806502} \approx 9.5 \quad (2)$$

Questo scostamento rispetto al fattore ideale è causato dagli overhead di gestione della memoria e dalle inefficienze intrinseche nell'accesso sequenziale agli elementi di matrici di grandi dimensioni (ad esempio per via dei *cache miss*).

L'algoritmo basato su libreria ottimizzata, invece, dimostra una scalabilità eccezionale. Al crescere di N , l'efficienza della libreria scava un divario incolmabile: per $N = 256$, la routine ottimizzata conclude il calcolo in circa 1.29 milisecondi, risultando oltre ***1320 volte più rapida*** della controparte manuale (1.715 secondi).

Un'eccezione interessante si nota per $N = 8$, dove il tempo della libreria (1.833677×10^{-3} s) risulta marcatamente anomalo ed è superiore perfino a quello registrato per le dimensioni successive fino a $N = 128$. Questo fenomeno è dovuto ai costi fissi di inizializzazione (*warm-up*) della libreria scientifica, come l'allocazione dinamica delle strutture, la pianificazione dell'algoritmo ottimale e il pre-calcolo dei fattori rotazionali. Per dimensioni estremamente ridotte ($N = 8$), l'overhead di chiamata della funzione supera di gran lunga il tempo di

calcolo effettivo, per poi stabilizzarsi e mostrare il reale andamento asintotico solo a partire da $N = 16$.

4 parte 2

Dopo aver integrato la versione ottimizzata della trasformata (*Fast DCT2*) nel modulo di compressione, il software è stato testato sul dataset di immagini `.bmp` in toni di grigio fornite. L'interfaccia grafica sviluppata consente di variare in tempo reale l'ampiezza del blocco F e la soglia di taglio d , mostrando l'immagine originale affiancata a quella compressa al fine di valutarne gli artefatti visivi. Dall'analisi comparativa dei risultati emergono differenze nette nel comportamento dell'algoritmo a seconda della natura geometrica del pattern processato.

4.0.1 Immagini Artificiali a Scacchiera e Gradienti

La compressione mostra gli effetti più distruttivi e innaturali sulle immagini geometriche artificiali (quali `20x20.bmp`, `40x40.bmp`, fino a `640x640.bmp`).

- **Perché l'algoritmo fa estrema fatica:** Dal punto di vista dell'analisi di Fourier, una transizione netta e istantanea dal nero totale (0) al bianco totale (255) — come il bordo di un quadrato della scacchiera — rappresenta una discontinuità a gradino (un segnale ad alta frequenza). Per ricostruire matematicamente questo sbalzo netto attraverso una combinazione di funzioni coseno, la DCT2 necessita di *tutti* i suoi coefficienti, specialmente di quelli ad altissima frequenza posizionati nell'angolo inferiore destro della matrice.
- **Artefatti visivi riscontrati:** Non appena si imposta una soglia $d < 2F - 2$ (eliminando le alte frequenze), i bordi netti dei quadrati si sfocano e compare il cosiddetto *fenomeno di Gibbs* o *ringing artifact*, ovvero vistose ondulazioni e aloni grigiastri attorno ai confini originariamente netti. Se il blocco F non è perfettamente allineato con la periodicità della scacchiera, l'insorgenza dell'“effetto blocco” (*blocking artifact*) altera completamente la geometria dell'immagine.
- **Il caso del gradiente:** Nell'immagine `gradient.bmp`, al contrario, la transizione è morbida e graduale. Trattandosi di variazioni a bassissima frequenza, l'algoritmo riesce a preservare un'ottima fedeltà visiva anche con valori di d molto bassi, poiché l'informazione strutturale è quasi interamente concentrata nei primissimi coefficienti in alto a sinistra della matrice (componente DC e prime armoniche).

4.0.2 Immagini Naturali e Fotografiche

Il comportamento cambia radicalmente quando l'algoritmo elabora immagini reali come `bridge.bmp`, `cathedral.bmp` e `deer.bmp`.

- **Perché l'algoritmo lavora meglio:** Le immagini fotografiche sono caratterizzate da un'alta *correlazione spaziale*; i pixel vicini tendono ad avere valori di luminosità molto simili e i dettagli finissimi (alte frequenze) spesso si confondono con il rumore di fondo o con texture naturali (*e.g.*, la pietra della cattedrale, il pelo del cervo o il cielo notturno del ponte). Di conseguenza, la stragrande maggioranza dell'energia informativa è concentrata nelle basse frequenze.
- **Analisi specifica del dataset:**
 - `gradient.bmp` e `deer.bmp`: In queste immagini sono presenti ampie zone a variazione sfumata o sfondo scuro uniforme. L'occhio umano fa molta fatica a percepire il taglio delle frequenze oltre la diagonale d in queste aree, rendendo la compressione estremamente efficiente e l'immagine ricostruita apparentemente identica all'originale.
 - `bridge.bmp` e `cathedral.bmp`: Sono i due casi in cui la differenza visiva diventa più evidente al calare di d . La struttura metallica del ponte e le intricate decorazioni barocche dell'interno della cattedrale contengono moltissime linee sottili e contrasti geometrici. Quando si riduce la soglia d , l'algoritmo "fa fatica" a mantenere la nitidezza di queste strutture: i dettagli minuti dell'architettura si fondono in una sfocatura omogenea e le impalcature del ponte perdono definizione, evidenziando la scomposizione in tasselli quadrati di dimensione F .

In conclusione, l'esperimento evidenzia il principio cardine della compressione *lossy* JPEG: l'algoritmo sfrutta i limiti della percezione visiva umana, la quale è fortemente insensibile alla perdita dei dettagli ad alta frequenza nelle immagini naturali, ma tollera molto male gli stessi tagli quando applicati a grafiche artificiali o a contorni ad alto contrasto.